



ITI · AI AGENTS DEVELOPMENT USING PYTHON TRACK

AI Agents Development & Agentic Automation

Eng. Fatma Elraey

Lab 2 — Build a Stateful LangGraph Agent with Human Approval

Field	Detail
Instructor	Fatma Elraey
Course	AI Agents Development (3-Day Course)
Session	Day 2 of 3 — Lab, 3 Hours
Focus	Hands-on construction of a stateful agent graph that persists threads and pauses before a sensitive action. A short CrewAI comparison is included at the end.

Before You Start

You should already understand the native tool-calling loop from Day 1. Today you will move the workflow state and routing into LangGraph so that memory, branching, and pause/resume behavior are explicit.

Today's Lab Plan

1. Set up LangGraph and Gemini
2. Define graph state and nodes
3. Compile with a checkpointer
4. Run separate persistent threads
5. Add an approval interrupt
6. Compare the same problem with CrewAI

```
pip install langgraph langchain-google-genai python-dotenv crewai --break-system-packages
```

✓ QUICK CHECK — Before You Start

Q1. What identifies one persisted conversation/run?
Answer: A thread_id in the graph configuration.

Q2. What lets a graph resume after an interrupt?
Answer: A checkpointer plus the same thread configuration.

01 Create the Project and Gemini Model

Create a small support-agent project and initialize Gemini through LangChain's Google integration.

- Create .env with GEMINI_API_KEY
- Keep the key out of Git
- Initialize ChatGoogleGenerativeAI
- Run one invoke() call to confirm the model connection

Why this matters: LangGraph controls the workflow; the chat model is one component inside that workflow.

Try this

```
import os
from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI

load_dotenv()
model = ChatGoogleGenerativeAI(
    model="gemini-3.7-flash",
    google_api_key=os.getenv("GEMINI_API_KEY"),
)

print(model.invoke("Reply with: model ready").content)
```

Expected result

```
# Instructor example — expected output
OUTPUT model ready
```

02 Define State and Nodes

Create a graph state that stores the user request, a draft response, and approval status. Add one model-backed node and one deterministic node.

- Use TypedDict for shared state
- Write draft_node(state)
- Write finalize_node(state)
- Make each node return only the state keys it updates

Why this matters: Explicit state contracts make agent workflows testable and easier to reason about than hidden variables inside a large loop.

Try this

```
from typing import TypedDict

class SupportState(TypedDict):
    request: str
    draft: str
    status: str

def draft_node(state: SupportState):
    prompt = f"Draft a concise support response for: {state['request']}"
    reply = model.invoke(prompt).content
    return {"draft": reply, "status": "drafted"}
```

```
def finalize_node(state: SupportState):
    return {"status": "ready"}
```

03 Build and Persist the Graph

Wire START → draft → finalize → END, then compile with an in-memory checkpointer and invoke it with a thread ID.

- Create StateGraph(SupportState)
- Add named nodes
- Add edges
- Compile with InMemorySaver
- Pass config={"configurable": {"thread_id": ...}}

Why this matters: Persistence is what later allows the same thread to pause, resume, or continue without losing its state.

Try this

```
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver

builder = StateGraph(SupportState)
builder.add_node("draft", draft_node)
builder.add_node("finalize", finalize_node)
builder.add_edge(START, "draft")
builder.add_edge("draft", "finalize")
builder.add_edge("finalize", END)

graph = builder.compile(checkpointer=InMemorySaver())
config = {"configurable": {"thread_id": "ticket-1001"}}

result = graph.invoke(
    {"request": "My order is three days late", "draft": "", "status": "new"},
    config,
)
print(result["status"])
```

Expected result

Instructor example — expected output

OUTPUT ready

PRO TIP Stretch: Run the same graph with thread IDs ticket-1002 and ticket-1003 and confirm that each run is isolated.

04 Add Human Approval Before a Sensitive Tool

Insert an approval node before a simulated send_email action. The graph should pause, expose the draft, and wait for a human decision.

- Import interrupt and Command
- Make approval_node call interrupt(...)
- Route True to send and False to cancel

- Compile with a checkpoint
- Invoke once to pause, then resume with `Command(resume=True)`

Why this matters: The agent can autonomously prepare an action while a human retains authority over the actual side effect.

Try this

```
from typing import Literal
from langgraph.types import Command, interrupt

def approval_node(state: SupportState) -> Command[Literal["send", "cancel"]]:
    approved = interrupt({
        "question": "Approve sending this message?",
        "draft": state["draft"],
    })
    return Command(goto="send" if approved else "cancel")

def send_node(state):
    print("SENT:", state["draft"])
    return {"status": "sent"}

def cancel_node(state):
    return {"status": "cancelled"}

# Rebuild graph with draft -> approval and routes to send/cancel.
# Then:
config = {"configurable": {"thread_id": "ticket-2001"}}
paused = graph.invoke(initial_state, config)
print(paused["__interrupt__"])
final = graph.invoke(Command(resume=True), config)
```

Expected result

Instructor example — expected output

OUTPUT First call pauses with an approval payload. The second call resumes the same thread and follows the approved route.

05

Test Reject and Edit Paths

Human approval should not be a single “yes” button in every workflow. Test rejection and an edited draft before resuming.

- Run a new thread and resume with `False`
- Confirm the sensitive action does not run
- Optionally interrupt with a dictionary containing `approved` + `edited_text`
- Update state with the edited text before send

Why this matters: A good HITL design gives reviewers enough control to understand, approve, edit, or reject sensitive actions.

Try this

```
# Reject example
config = {"configurable": {"thread_id": "ticket-2002"}}
graph.invoke(initial_state, config)
rejected = graph.invoke(Command(resume=False), config)
print(rejected["status"]) # cancelled
```

Expected result

Instructor example — expected output

OUTPUT cancelled

PRO TIP *Stretch: Change the interrupt payload so a reviewer can return an edited draft, then send the edited version only after approval.*

06

Mini Comparison: CrewAI

Recreate a simple two-role workflow in CrewAI so you can feel the difference in abstraction: LangGraph makes state transitions explicit; CrewAI makes team roles and tasks explicit.

- Create one analyst Agent
- Create one reviewer Agent
- Create sequential Tasks
- Kick off a Crew
- Compare the code to the graph you built

Why this matters: The right framework is the one that makes the intended workflow easiest to understand, test, and control.

Try this

```
from crewai import Agent, Task, Crew, Process

analyst = Agent(
    role="Support Analyst",
    goal="Draft a factual response",
    backstory="You inspect the customer request carefully.",
)
reviewer = Agent(
    role="Quality Reviewer",
    goal="Check the response for clarity and risk",
    backstory="You never approve unsupported claims.",
)

draft_task = Task(
    description="Draft a response for the supplied support request.",
    expected_output="A concise customer-facing draft.",
    agent=analyst,
)
review_task = Task(
    description="Review the draft and return a corrected final response.",
    expected_output="A final approved response.",
    agent=reviewer,
)

crew = Crew(
    agents=[analyst, reviewer],
    tasks=[draft_task, review_task],
    process=Process.sequential,
)
print(crew.kickoff())
```

PRO TIP *Stretch: Write two sentences explaining when you would choose this role/task abstraction over an explicit LangGraph state machine.*

Congratulations

You built a stateful agent architecture with persisted threads and a real human approval checkpoint. Those patterns are central to reliable long-running agent systems.